

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN



MÔN HỌC: TOÁN ỨNG DỤNG VÀ THỐNG KÊ

BÁO CÁO ĐỒ ÁN 2

DATA FITTING

Giảng viên lí thuyết: Dương Việt Hằng
Giảng viên thực hành: Võ Nam Thực Doan
Giảng viên thực hành: Lê Nhựt Nam
Trợ giảng: Đỗ Đức Hòa

Sinh viên thực hiện: Ngô Hoàng Minh - 24120381
Mai Thúc Hải Đăng - 24120276
Nguyễn Thành Dữ - 24120288
Đỗ Ngọc Hải - 24120300
Nguyễn Xuân Lộc - 24120369

Thành phố Hồ Chí Minh, tháng 4/2026

Mục lục

Bảng Phân Công Nhiệm Vụ	3
1 1. Phần 1: Lý Thuyết Data Fitting và Minh Họa	4
1.1 Hàm <code>ols_fit</code>	4
1.1.1 Cơ sở lý thuyết	4
1.1.2 Xây dựng hàm	5
1.2 Hàm <code>hat_matrix</code>	7
1.2.1 Cơ sở lý thuyết	7
1.2.2 Xây dựng hàm	9
1.3 Hàm <code>coef_inference()</code>	11
1.3.1 Cơ sở lý thuyết	11
1.3.2 Xây dựng hàm <code>coef_inference()</code>	12
1.4 Hồi quy Ridge (Ridge Regression)	13
1.4.1 Biểu đồ Ridge Trace	13
1.5 Kiểm định chéo K-Fold (K-Fold Cross-Validation)	14
1.5.1 Lựa chọn tham số λ	14
1.6 Hàm <code>model_metrics</code>	14
1.6.1 Cơ sở lý thuyết	14
1.6.2 Xây dựng hàm	15
1.7 Hàm <code>calculate_vif</code>	16
1.7.1 Cơ sở lý thuyết	16
1.7.2 Xây dựng hàm	16
1.8 Hàm <code>residual_analysis</code>	17
1.8.1 Phương pháp cài đặt	17
1.8.2 Mã nguồn cài đặt	17
1.8.3 Trực quan hóa và Phân tích kết quả	19
2 Phần 2: Ứng Dụng Data Fitting vào Dữ Liệu Thực Tế	21
2.1 Giới thiệu về bộ data	21
2.2 Khảo sát dữ liệu	22
2.3 Pipeline xử lý dữ liệu	28
2.3.1 Phân loại cột và Encoding	28
2.3.2 Điền Missing Value	28
2.3.3 Xử lý Outlier & Scaling	28
2.3.4 Feature Engineering (Kỹ thuật đặc trưng)	30
2.4 Quy trình xử lý dữ liệu	30
2.5 Chẩn đoán đa cộng tuyến và Lựa chọn đặc trưng (Feature Selection)	31

2.5.1	Chẩn đoán đa cộng tuyến bằng VIF	31
2.5.2	Lựa chọn biến bằng P-value (Backward Elimination)	31
2.6	Mô hình Ridge và Lasso	32
2.6.1	Cài đặt thuật toán Lasso	32
2.6.2	Quá trình chọn λ với K-Fold Cross-Validation	33
2.7	Kỹ thuật nâng cao: Kernel Ridge và Bayesian Linear Regression	34
2.7.1	Kernel Ridge Regression (KRR)	34
2.7.2	Bayesian Linear Regression (BLR)	34
2.8	Đánh giá và So sánh Mô hình	34
2.8.1	Lựa chọn mô hình tối ưu và Giải thích	35
2.9	Đánh giá độ quan trọng của đặc trưng (Feature Importance)	36
2.10	Chẩn đoán phần dư cho mô hình OLS trên dữ liệu thực tế	37
3	Kết luận	38

Bảng Phân Công Nhiệm Vụ

Thành viên	Nhiệm vụ đảm nhận
Minh	Part 1: Tích hợp Notebook, mô phỏng Monte Carlo, hàm <code>coef_inference</code> . Part 2: Kiến trúc Pipeline & Kỹ thuật nâng cao (Kernel Ridge, Bayesian LR).
Dự	Part 1: Nền tảng OLS (<code>ols_fit</code> , <code>hat_matrix</code>). Part 2: Tiền xử lý dữ liệu, Xử lý Missing Values, Chuẩn hóa, Encoding và Feature Engineering.
Lộc	Part 1: Đo lường lỗi & Đa cộng tuyến (<code>model_metrics</code> , <code>vif</code>). Part 2: Khảo sát dữ liệu (EDA), Xử lý Outliers, Chẩn đoán đa cộng tuyến và Chọn biến.
Đăng	Part 1: Chính quy hóa (<code>ridge_fit</code>) & Đánh giá (<code>kfold_cv</code>). Part 2: Huấn luyện mô hình cơ sở, Chính quy hóa (Ridge/Lasso) và So sánh mô hình.
Hải	Part 1: Phân tích trực quan (<code>residual_plots</code>), Biên tập Báo cáo tổng. Part 2: Trực quan hóa dữ liệu (Plots) và Tổng hợp, biên tập Báo cáo LaTeX.

Bảng 1: Phân công công việc nhóm (Part 1 & Part 2)

1. 1. Phần 1: Lý Thuyết Data Fitting và Minh Họa

1.1. Hàm `ols_fit`

1.1.1. Cơ sở lý thuyết

Phương pháp Bình phương Cực tiểu (Ordinary Least Squares - OLS) là kỹ thuật phổ biến nhất để ước lượng các tham số trong mô hình hồi quy tuyến tính cổ điển. Giả thuyết mô hình hồi quy tuyến tính đa biến có dạng:

$$y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \cdots + \beta_p x_{ip} + \varepsilon_i, \quad i = 1, 2, \dots, n \quad (1)$$

Trong đó:

- y_i là giá trị thực tế của biến phụ thuộc đối với quan sát thứ i .
- x_{ij} là giá trị của biến độc lập thứ j đối với quan sát thứ i .
- $\beta_0, \beta_1, \dots, \beta_p$ là các hệ số hồi quy cần ước lượng.
- ε_i là thành phần sai số ngẫu nhiên (nhiều).

Dưới dạng ma trận, mô hình được viết gọn lại là:

$$y = X\beta + \varepsilon \quad (2)$$

Với:

- $y = [y_1, y_2, \dots, y_n]^T \in \mathbb{R}^{n \times 1}$ là vector biến phụ thuộc.
- $X \in \mathbb{R}^{n \times (p+1)}$ là ma trận thiết kế (Design Matrix), cột đầu tiên chứa toàn bộ giá trị 1 tương ứng với hệ số chặn (intercept β_0):

$$X = \begin{bmatrix} 1 & x_{11} & x_{12} & \dots & x_{1p} \\ 1 & x_{21} & x_{22} & \dots & x_{2p} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n1} & x_{n2} & \dots & x_{np} \end{bmatrix} \quad (3)$$

- $\beta = [\beta_0, \beta_1, \dots, \beta_p]^T \in \mathbb{R}^{(p+1) \times 1}$ là vector tham số cần ước lượng.
- $\varepsilon = [\varepsilon_1, \varepsilon_2, \dots, \varepsilon_n]^T \in \mathbb{R}^{n \times 1}$ là vector sai số ngẫu nhiên.

Mục tiêu của phương pháp OLS là tìm ước lượng $\hat{\beta}$ sao cho tổng bình phương phần dư (Residual Sum of Squares - RSS) đạt giá trị nhỏ nhất:

$$RSS(\beta) = \|\varepsilon\|_2^2 = \|y - X\beta\|_2^2 = (y - X\beta)^T(y - X\beta) = y^T y - 2\beta^T X^T y + \beta^T X^T X \beta \quad (4)$$

Để tìm cực trị của $RSS(\beta)$, ta lấy đạo hàm riêng của RSS theo vector tham số β và thiết lập phương trình bằng 0:

$$\nabla_{\beta} RSS(\beta) = -2X^T y + 2X^T X \beta = 0 \quad (5)$$

Điều này dẫn đến hệ phương trình chuẩn (Normal Equations):

$$X^T X \beta = X^T y \quad (6)$$

Dưới giả thiết Gauss-Markov GM2 (Không hoàn hảo đa cộng tuyến), ma trận thiết kế X có đầy đủ hạng cột, nghĩa là các cột của X độc lập tuyến tính. Do đó, ma trận vuông $X^T X \in \mathbb{R}^{(p+1) \times (p+1)}$ khả nghịch. Nghiệm đóng duy nhất của bài toán OLS là:

$$\hat{\beta}_{OLS} = (X^T X)^{-1} X^T y \quad (7)$$

Sau khi tìm được vector hệ số $\hat{\beta}$, giá trị dự báo tương ứng là $\hat{y} = X\hat{\beta}$, và vector phần dư là $e = y - \hat{y}$.

Để đánh giá sai số ngẫu nhiên, ta ước lượng phương sai của nhiễu σ^2 . Một ước lượng không chệch của phương sai nhiễu σ^2 được xác định bởi công thức:

$$\hat{\sigma}^2 = \frac{RSS}{n - (p + 1)} = \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{n - k} \quad (8)$$

Trong đó $k = p + 1$ là số lượng tham số trong mô hình (bao gồm cả hệ số chặn). Mẫu số $n - k$ chính là bậc tự do của phần dư.

1.1.2. Xây dựng hàm

Hàm `ols_fit` được lập trình hoàn toàn từ đầu (from scratch) mà không sử dụng các hàm đại số tuyến tính của thư viện NumPy. Thuật toán sử dụng các hàm hỗ trợ trong `helper_function.py` để thực hiện các phép toán chuyển vị (`transpose_matrix`), nhân ma trận (`matmul`), nhân ma trận với vector (`matvec`) và nghịch đảo ma trận bằng thuật toán Gauss-Jordan tìm pivot (`invert_matrix`).

Quy trình tính toán cụ thể gồm các bước:

1. **Chuẩn bị dữ liệu:** Chèn thêm cột chứa toàn bộ giá trị 1 vào đầu ma trận đặc trưng X bằng hàm `add_intercept` và ép kiểu vector mục tiêu y về dạng danh sách một chiều bằng `to_1d_list`.
2. **Kiểm tra điều kiện:** Xác nhận kích thước dữ liệu hợp lệ (số mẫu n phải khớp giữa X và y , đồng thời $n > k$ để đảm bảo đủ bậc tự do).
3. **Tính toán các đại lượng trung gian:**
 - Tính chuyển vị của X : $X^T \leftarrow \text{transpose_matrix}(X_mat)$.
 - Tính tích $X^T X$: $X^T X \leftarrow \text{matmul}(Xt, X_mat)$.
 - Tìm nghịch đảo $(X^T X)^{-1}$: $(X^T X)^{-1} \leftarrow \text{invert_matrix}(XtX)$. Phương pháp khử Gauss-Jordan kết hợp tìm phần tử xoay (pivot) giúp đảm bảo độ chính xác số học và phát hiện sớm hiện tượng đa cộng tuyến hoàn hảo (nếu pivot lớn nhất ≈ 0).
 - Tính tích $X^T y$: $X^T y \leftarrow \text{matvec}(Xt, y_vec)$.
4. **Tìm nghiệm OLS:** Tính vector hệ số $\hat{\beta} = (X^T X)^{-1} X^T y$ bằng hàm nhân ma trận với vector `matvec`.
5. **Ước lượng phương sai nhiễu $\hat{\sigma}^2$:** Tính các giá trị dự báo $\hat{y} = X\hat{\beta}$, suy ra vector phần dư $e_i = y_i - \hat{y}_i$. Sau đó tính tổng bình phương phần dư $RSS = \sum e_i^2$ và chia cho bậc tự do $n - k$.

Dưới đây là mã nguồn chi tiết của hàm `ols_fit`:

```

1 def ols_fit(X, y):
2     X_mat = add_intercept(X)
3     y_vec = to_1d_list(y)
4     n = len(X_mat)
5     k = len(X_mat[0])
6     if n != len(y_vec):
7         raise ValueError(f"Số hàng X ({n}) và chiều dài y ({len(y_vec)}) không khớp.")
8     if n <= k:
9         raise ValueError(f"Can n > p+1. Hien n={n}, p+1={k}.")
10    # Buoc 1: Xt
11    Xt = transpose_matrix(X_mat)
12    # Buoc 2: XtX
13    XtX = matmul(Xt, X_mat)
14    # Buoc 3: (XtX)^-1
15    XtX_inv = invert_matrix(XtX)
16    # Buoc 4: Xty
17    Xty = matvec(Xt, y_vec)
18    # Buoc 5: beta_hat = (XtX)^-1 Xty
19    beta_hat = matvec(XtX_inv, Xty)
20    # Tinh sigma2_hat = RSS / (n - p - 1)
21    y_hat = matvec(X_mat, beta_hat)
22    residuals = [y_vec[i] - y_hat[i] for i in range(n)]
23    rss = sum(r * r for r in residuals)
24    sigma2_hat = rss / (n - k)
25    return beta_hat, sigma2_hat

```

1.2. Hàm `hat_matrix`

1.2.1. Cơ sở lý thuyết

Ma trận chiếu (Projection Matrix), thường được gọi là ma trận mũ (Hat Matrix) ký hiệu là H , đóng vai trò quan trọng trong việc phân tích các tính chất hình học của mô hình hồi quy tuyến tính. Ma trận này ánh xạ vector các giá trị thực tế của biến phụ thuộc y thành vector các giá trị dự báo $\hat{y}$:

$$\hat{y} = Hy \quad (9)$$

Từ nghiệm đóng của OLS $\hat{\beta}_{OLS} = (X^T X)^{-1} X^T y$, ta có:

$$\hat{y} = X \hat{\beta}_{OLS} = X (X^T X)^{-1} X^T y \quad (10)$$

Do đó, ma trận mũ $H \in \mathbb{R}^{n \times n}$ được xác định bằng công thức:

$$H = X (X^T X)^{-1} X^T \quad (11)$$

Ma trận Hat có các tính chất đại số tuyến tính vô cùng đặc biệt:

- **Tính đối xứng (Symmetric):** Ma trận H bằng ma trận chuyển vị của chính nó, $H^T = H$.

$$\begin{aligned} H^T &= (X (X^T X)^{-1} X^T)^T \\ &= (X^T)^T ((X^T X)^{-1})^T X^T \\ &= X (X^T X)^{-1} X^T = H \end{aligned} \quad (12)$$

- **Tính lũy đẳng (Idempotent):** Áp dụng phép chiếu liên tiếp hai hay nhiều lần không làm thay đổi kết quả chiếu, $H^2 = H$.

$$\begin{aligned} H^2 &= H \cdot H = (X (X^T X)^{-1} X^T) (X (X^T X)^{-1} X^T) \\ &= X (X^T X)^{-1} (X^T X) (X^T X)^{-1} X^T \\ &= X (X^T X)^{-1} X^T = H \end{aligned} \quad (13)$$

Tính chất lũy đẳng này đồng nghĩa với việc chiếu một vector đã nằm trong không gian cột của X (như $\hat{y}$) vào không gian cột của X thì vẫn giữ nguyên chính nó.

- **Hạng (Rank) và Vết (Trace):** Hạng của một ma trận chiếu trực giao bằng vết của nó (tổng các phần tử trên đường chéo chính), và bằng số chiều của không gian con mà nó chiếu lên (tức là số cột độc lập tuyến tính của X):

$$\begin{aligned} \text{rank}(H) &= \text{trace}(H) \\ &= \text{trace}(X (X^T X)^{-1} X^T) \\ &= \text{trace}(X^T X (X^T X)^{-1}) \\ &= \text{trace}(I_{p+1}) = p + 1 = k \end{aligned} \quad (14)$$

- **Giá trị riêng (Eigenvalues):** Vì H là ma trận lũy đẳng, các giá trị riêng λ của nó chỉ có thể nhận giá trị 0 hoặc 1. Thật vậy, nếu v là vector riêng của H ứng

với giá trị riêng λ ($v \neq 0$), ta có:

$$Hv = \lambda v \implies H^2v = H(Hv) = \lambda Hv = \lambda^2 v \quad (15)$$

Do $H^2 = H$ nên:

$$\lambda^2 v = \lambda v \implies \lambda(\lambda - 1)v = 0 \implies \lambda = 0 \quad \text{hoặc} \quad \lambda = 1 \quad (16)$$

Số lượng các giá trị riêng bằng 1 của H đúng bằng $\text{rank}(H) = k$, và số lượng các giá trị riêng bằng 0 là $n - k$.

- **Độ ảnh hưởng (Leverage):** Các phần tử trên đường chéo chính của H , ký hiệu là $h_{ii} = H_{ii}$, đại diện cho độ ảnh hưởng (leverage) của quan sát thứ i lên giá trị dự báo của chính nó $\hat{y}_i$. Do H là ma trận đối xứng và lũy đẳng, ta có:

$$h_{ii} = \sum_{j=1}^n H_{ij}^2 = h_{ii}^2 + \sum_{j \neq i} H_{ij}^2 \implies h_{ii} \geq h_{ii}^2 \implies 0 \leq h_{ii} \leq 1 \quad (17)$$

Tổng các leverage của toàn bộ dữ liệu bằng số lượng tham số trong mô hình: $\sum_{i=1}^n h_{ii} = k$. Những điểm dữ liệu có giá trị h_{ii} cao ($h_{ii} > 2k/n$ hoặc $3k/n$) được xem là các điểm đòn bẩy (high leverage points), có khả năng kéo đường hồi quy về phía chúng một cách mạnh mẽ.

1.2.2. Xây dựng hàm

Hàm `hat_matrix` thực hiện tính toán ma trận chiều H trực tiếp từ ma trận thiết kế X và xác thực bằng số tính chất lũy đẳng của nó.

Các bước thực hiện trong hàm:

1. **Chuẩn bị dữ liệu:** Gọi hàm `add_intercept(X)` để đảm bảo mô hình có chứa hệ số chặn, từ đó thu được ma trận X mở rộng gồm n dòng và $k = p + 1$ cột.

2. **Tính toán ma trận H :**

- Tính chuyển vị X^T : $X^T \leftarrow \text{transpose_matrix}(X_mat)$.
- Tính tích ma trận $X^T X$: $X^T X \leftarrow \text{matmul}(Xt, X_mat)$.
- Tính nghịch đảo của $(X^T X)$ bằng khử Gauss-Jordan: $(X^T X)^{-1} \leftarrow \text{invert_matrix}(XtX)$.
- Tính tích ma trận trung gian $M_1 = X(X^T X)^{-1}$: $M_1 \leftarrow \text{matmul}(X_mat, XtX_inv)$.
- Tính ma trận Hat $H = M_1 X^T$: $H \leftarrow \text{matmul}(M1, Xt)$.

3. Kiểm tra tính lũy đẳng (Idempotent verification):

- Tính $H^2 = H \cdot H$ bằng hàm `matmul(H, H)`.
- So sánh từng phần tử H_{ij}^2 với H_{ij} . Nếu tồn tại bất kỳ cặp phần tử nào có độ lệch tuyệt đối vượt quá ngưỡng sai số nhỏ (10^{-9}), hệ thống kết luận ma trận không thỏa mãn tính lũy đẳng:

$$|H_{ij}^2 - H_{ij}| > 10^{-9} \quad (18)$$

- Trả về ma trận H cùng với biến logic `is_idempotent` (nhận giá trị `True` nếu thỏa mãn tính chất lũy đẳng và `False` trong trường hợp ngược lại).

Dưới đây là mã nguồn chi tiết của hàm `hat_matrix`:

```

1 def hat_matrix(X):
2     X_mat = add_intercept(X)
3     n = len(X_mat)
4
5     Xt = transpose_matrix(X_mat)
6
7     # Buoc 2: XtX
8     XtX = matmul(Xt, X_mat)
9
10    # Buoc 3: (XtX)^-1
11    XtX_inv = invert_matrix(XtX)
12
13    # Buoc 4: H = X (XtX)^-1 Xt
14    M1 = matmul(X_mat, XtX_inv)
15    H = matmul(M1, Xt)
16
17    # Kiem tra tinh Idempotent (H^2 = H)
18    H2 = matmul(H, H)
19    is_idempotent = True
20    for i in range(n):
21        for j in range(n):
22            if abs(H2[i][j] - H[i][j]) > 1e-9:
23                is_idempotent = False
24                break
25        if not is_idempotent:
26            break
27
28    return H, is_idempotent

```

1.3. Hàm `coef_inference()`

1.3.1. Cơ sở lí thuyết

Theo **định lý Gauss-Markov** (dưới các giả thiết GM1-GM4), ma trận hiệp phương sai của vector hệ số OLS β được tính bằng

$$\text{Var}(\hat{\beta}_{OLS}|X) = \sigma^2(X^T X)^{-1}$$

Trong đó, phương sai của từng hệ số riêng lẻ $\hat{\beta}_j$ chính là các phần tử nằm trên đường

chéo chính của ma trận này **Standar Errors** được tính bằng công thức

$$SE(\beta_j) = \sqrt{\hat{\sigma}^2[(X^T X)^{-1}]_{jj}}$$

Trong đó: β_j là cột j của β , σ^2 được lấy từ hàm `ols_fit()`

Về phần **Giá trị kiểm định (t-statistics)**: Để kiểm định xem một biến đặc trưng có thực sự đóng góp vào mô hình hay không, ta đặt giả thuyết không $H_0 : \beta_j = 0$. Dưới giả thiết nhiễu phân phối chuẩn (GM5), tỷ số giữa hệ số ước lượng và sai số chuẩn của nó tuân theo phân phối Student:

$$t_j = \frac{\hat{\beta}_j}{\hat{\sigma} \sqrt{[(X^T X)^{-1}]_{jj}}} \sim t_{n-p-1}$$

Về **Bậc tự do**: Bậc tự do của mô hình xác định hình dáng của phân phối Student. Công thức là $df = n - p - 1$.

- n : Số lượng quan trắc (số hàng của ma trận dữ liệu).
- p : Số lượng biến độc lập (không tính hệ số chặn).

Về **Khoảng tin cậy 95%** Từ phân phối Student, ta có thể tìm được một giá trị tới hạn $t_{critical}$ (hoặc $t_{\alpha/2}$) sao cho phần diện tích ở giữa hai đuôi phân phối chiếm đúng 95%. Khoảng tin cậy cho hệ số β_j thực sự trong tổng thể sẽ là:

$$\beta_j \in [\hat{\beta}_j - t_{critical} \times SE(\hat{\beta}_j), \hat{\beta}_j + t_{critical} \times SE(\hat{\beta}_j)]$$

1.3.2. Xây dựng hàm `coef_inference()`

Vì không dùng thư viện tính tích phân, hàm `get_t_critical(df)` đóng vai trò như việc tra bảng phân phối Student. Nhóm em khai báo như sau

```
1 t_table = {
2     1: 12.706, 2: 4.303, 3: 3.182, 4: 2.776, 5: 2.571,
3     6: 2.447, 7: 2.365, 8: 2.306, 9: 2.262, 10: 2.228,
4     12: 2.179, 15: 2.131, 20: 2.086, 25: 2.060, 30: 2.042,
5     40: 2.021, 50: 2.009, 60: 2.000, 80: 1.990, 100: 1.984
6 }
```

Tiếp theo, hàm `coef_inference()` sử dụng `t_table` ở trên để lần lượt tính các đại lượng cần thiết, gồm standard errors, t-statistics, giá trị tới hạn `t_critical`, khoảng tin cậy 95% và mức ý nghĩa ở ngưỡng 5%. Kết quả trả về như sau:

```

1 return {
2     "df": df,
3     "t_critical": t_critical,
4     "Standard_Errors": Standard_Errors,
5     "t_stats": t_list,
6     "Confidence_Intervals": conf_intervals,
7     "Significance_at_5pct": significance
8 }

```

1.4. Hồi quy Ridge (Ridge Regression)

Hồi quy Ridge là một kỹ thuật chính quy hóa được sử dụng để giải quyết vấn đề đa cộng tuyến và quá khớp. Khác với OLS truyền thống, Ridge Regression thêm một thành phần phạt L_2 vào hàm mất mát để hạn chế độ lớn của các hệ số hồi quy.

Hàm mất mát của Ridge Regression được định nghĩa như sau:

$$J(\beta) = \sum_{i=1}^n (y_i - x_i^T \beta)^2 + \lambda \sum_{j=1}^p \beta_j^2 = \|y - X\beta\|^2 + \lambda \|\beta\|_2^2 \quad (19)$$

Trong đó:

- $\lambda \geq 0$ là tham số điều chỉnh.
- Khi $\lambda = 0$, mô hình trở về OLS.
- Khi $\lambda \rightarrow \infty$, các hệ số β (trừ hệ số chặn β_0) sẽ bị ép về gần 0.

Nghiệm của bài toán tối ưu (dạng đóng) là:

$$\hat{\beta}_{ridge} = (X^T X + \lambda I')^{-1} X^T y \quad (20)$$

Trong đó I' là ma trận đơn vị đã được điều chỉnh (thường đặt phần tử $I'_{0,0} = 0$ để không thực hiện chính quy hóa cho hệ số chặn β_0).

1.4.1. Biểu đồ Ridge Trace

Ridge Trace là biểu đồ thể hiện sự thay đổi của các hệ số hồi quy β_j theo các giá trị khác nhau của λ . Biểu đồ này giúp trực quan hóa quá trình "co rút" của các biến và hỗ trợ việc lựa chọn λ phù hợp.

1.5. Kiểm định chéo K-Fold (K-Fold Cross-Validation)

Để đánh giá hiệu năng của mô hình một cách khách quan và lựa chọn tham số λ tối ưu, chúng ta sử dụng kỹ thuật K-Fold Cross-Validation.

Quy trình thực hiện bao gồm các bước:

1. Xáo trộn ngẫu nhiên tập dữ liệu.
2. Chia tập dữ liệu thành k phần (folds) có kích thước bằng nhau.
3. Với mỗi fold thứ i :
 - Sử dụng fold i làm tập kiểm thử.
 - Sử dụng $k - 1$ fold còn lại làm tập huấn luyện.
 - Huấn luyện mô hình và tính sai số bình phương trung bình MSE_i .
4. Tính sai số kiểm định chéo trung bình:

$$CV_{(k)} = \frac{1}{k} \sum_{i=1}^k MSE_i \quad (21)$$

1.5.1. Lựa chọn tham số λ

Chúng ta thực hiện K-Fold CV trên một lưới các giá trị λ khác nhau. Giá trị λ cho $CV_{(k)}$ nhỏ nhất sẽ được chọn làm tham số tối ưu cho mô hình cuối cùng.

1.6. Hàm `model_metrics`

1.6.1. Cơ sở lý thuyết

Để đánh giá độ phù hợp của mô hình hồi quy tuyến tính, chúng ta sử dụng các chỉ số thống kê quan trọng dựa trên sự phân rã tổng bình phương các sai lệch. Các chỉ số cốt lõi bao gồm:

- **RSS (Residual Sum of Squares):** Tổng bình phương các phần dư, đo lường phần biến thiên của biến phụ thuộc không được giải thích bởi mô hình.

$$RSS = \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (22)$$

- **TSS (Total Sum of Squares):** Tổng bình phương tổng thể, đo lường tổng biến thiên của biến phụ thuộc xung quanh giá trị trung bình.

$$TSS = \sum_{i=1}^n (y_i - \bar{y})^2 \quad (23)$$

- **Hệ số xác định R^2 :** Tỷ lệ biến thiên của biến phụ thuộc được giải thích bởi các biến độc lập trong mô hình.

$$R^2 = 1 - \frac{RSS}{TSS} \quad (24)$$

- **Hệ số xác định hiệu chỉnh $\bar{R}^2$:** Hiệu chỉnh R^2 để tránh việc tăng ảo khi thêm biến vào mô hình, có tính đến bậc tự do.

$$\bar{R}^2 = 1 - (1 - R^2) \frac{n - 1}{n - p - 1} \quad (25)$$

- **Kiểm định F (F-test):** Đánh giá ý nghĩa tổng thể của mô hình. Giả thuyết H_0 cho rằng tất cả các hệ số hồi quy (trừ hệ số chặn) đều bằng 0.

$$F = \frac{(TSS - RSS)/p}{RSS/(n - p - 1)} \quad (26)$$

1.6.2. Xây dựng hàm

Hàm `model_metrics` được xây dựng tập trung vào việc tính toán tường minh các đại lượng sai lệch bình phương.

Tính toán RSS và TSS: Sử dụng *list comprehension* kết hợp với hàm `sum()` để tối ưu hóa hiệu suất và giữ mã nguồn ngắn gọn:

Listing 1: Tính toán RSS và TSS

```

1 # RSS: Residual Sum of Squares
2 rss = sum((yt - yp)**2 for yt, yp in zip(y_true, y_pred))
3
4 # TSS: Total Sum of Squares
5 tss = sum((yt - y_mean)**2 for yt in y_true)
```

Đoạn mã trên sử dụng `zip` để ghép cặp giá trị thực và giá trị dự báo, sau đó tính bình phương hiệu số.

Tính toán các chỉ số phái sinh: Sau khi có RSS và TSS, các chỉ số còn lại được tính theo công thức lý thuyết, kèm theo kiểm tra điều kiện để tránh lỗi chia cho 0:

Listing 2: Tính toán R2 và F-statistic

```

1 # R^2: Coefficient of Determination
2 r2 = 1 - (rss / tss) if tss != 0 else 0
3
4 # F-statistic
5 if rss == 0:
6     f_stat = float('inf')
7 else:
8     f_stat = ((tss - rss) / p) / (rss / (n - p - 1))

```

1.7. Hàm calculate_vif

1.7.1. Cơ sở lý thuyết

Hệ số phóng đại phương sai (Variance Inflation Factor - VIF) đo lường mức độ ảnh hưởng của đa cộng tuyến. VIF của biến X_j được tính bằng:

$$VIF_j = \frac{1}{1 - R_j^2} \quad (27)$$

Trong đó R_j^2 là hệ số xác định khi hồi quy biến X_j theo tất cả các biến độc lập khác trong mô hình.

1.7.2. Xây dựng hàm

Hàm `calculate_vif` thực hiện quy trình hồi quy lặp đi lặp lại cho từng đặc trưng.

Thiết lập hồi quy phụ: Với mỗi cột, chúng ta tách dữ liệu và chuẩn bị ma trận thiết kế cho mô hình OLS phụ:

Listing 3: Chuẩn bị dữ liệu cho hồi quy phụ

```

1 for feature in features:
2     # Biến mục tiêu phụ là feature hiện tại
3     y = X_df[feature].tolist()
4
5     # Các biến độc lập phụ là các feature còn lại
6     other_features = [f for f in features if f != feature]
7     X_others = X_df[other_features]
```

Giải OLS và tính VIF: Sử dụng hàm `ols_fit` để tìm nghiệm tối ưu, sau đó tính toán VIF dựa trên kết quả R^2 thu được từ hàm `model_metrics`:

Listing 4: Tính toán VIF từ nghiệm đóng

```

1 # Sử dụng ols_fit để giải hồi quy phụ
2 beta_hat, _ = ols_fit(X_others, y)
3
4 # Lấy R2 từ hàm metrics
5 metrics = model_metrics(y, y_pred_aux, len(other_features))
6 r2_aux = metrics["R2"]
7
8 # Tính VIF
9 vif = 1 / (1 - r2_aux) if r2_aux < 1 else float('inf')
```

1.8. Hàm residual_analysis

1.8.1. Phương pháp cài đặt

- **Residuals:** $\hat{\epsilon}_i = y_i - \hat{y}_i$, dùng để đo lường sai số dự báo
- **Standardized Residuals:** $r_i = \frac{\hat{\epsilon}_i}{\hat{\sigma}\sqrt{1-h_{ii}}}$, trong đó h_{ii} là các phần tử trên đường chéo của ma trận Hat (H)
- **Cook's Distance:** $D_i = \frac{r_i^2}{p+1} \left(\frac{h_{ii}}{1-h_{ii}} \right)$, dùng để xác định các quan sát có ảnh hưởng lớn.

1.8.2. Mã nguồn cài đặt

Listing 5: Cài đặt hàm residual_analysis

```
1
2 def residual_analysis(X, y, beta_hat):
3     n = len(y)
4     p = len(X[0])
5     # Tính y_hat = X * beta_hat
6     y_hat = [sum(X[i][j] * beta_hat[j] for j in range(p + 1))
7               for i in range(n)]
8     residuals = [y[i] - y_hat[i] for i in range(n)]
9
10    # Lấy đường chéo h_ii từ hat_matrix
11    H, _ = hat_matrix(X)
12    h_ii = [H[i][i] for i in range(n)]
13
14    # Tính Standardized Residuals
15    rss = sum(r**2 for r in residuals)
16    sigma2 = rss / (n - p - 1)
17    std_residuals = [residuals[i] / math.sqrt(sigma2 * (1 -
18    h_ii[i])) for i in range(n)]
19
20    # Tính Cook's Distance
21    cooks_d = [(std_residuals[i]**2 / (p + 1)) * (h_ii[i] / (1
22    - h_ii[i])) for i in range(n)]
23
24    # Tiến hành vẽ 4 biểu đồ
25    fig, axs = plt.subplots(2, 2, figsize=(14, 10))
26    fig.suptitle('Residual Analysis Diagnostics', fontsize=16)
27
28    # Biểu đồ 1: Residuals vs Fitted
29    axs[0, 0].scatter(y_hat, residuals, alpha=0.6, edgecolors='
30    w', label='Residuals')
31    axs[0, 0].axhline(0, color='red', linestyle='--', lw=2,
32    label='Zero residual line')
33    axs[0, 0].set_title('Residuals vs Fitted')
34    axs[0, 0].set_xlabel('Fitted values')
35    axs[0, 0].set_ylabel('Residuals')
36    axs[0, 0].legend()
37
38    # Biểu đồ 2: Normal Q-Q
39    stats.probplot(std_residuals, dist="norm", plot=axs[0, 1])
40    axs[0, 1].get_lines()[1].set_color('red')
```

```

36     axs[0, 1].set_title('Normal_Q-Q')
37     axs[0, 1].set_xlabel('Theoretical_quantiles')
38     axs[0, 1].set_ylabel('Standardized_Residuals')
39     axs[0, 1].legend(['Observed_residuals', 'Normal_reference_
    line'])
40
41     # Bieu do 3: Scale-Location
42     sqrt_std_residuals = [math.sqrt(abs(sr)) for sr in
    std_residuals]
43     axs[1, 0].scatter(y_hat, sqrt_std_residuals, alpha=0.6,
    edgecolors='w', label='Scale-location_points')
44     axs[1, 0].set_title('Scale-Location')
45     axs[1, 0].set_xlabel('Fitted_values')
46     axs[1, 0].set_ylabel('$\sqrt{|Standardized\_Residuals|}$')
47     axs[1, 0].legend()
48
49     # Bieu do 4: Cook's Distance
50     markerline, stemlines, baseline = axs[1, 1].stem(range(n),
    cooks_d, markerfmt="," , linefmt="b-", basefmt="_")
51     markerline.set_label("Cook's_distance")
52     axs[1, 1].set_title("Cook's_Distance")
53     axs[1, 1].set_xlabel('Observation_Index')
54     axs[1, 1].set_ylabel("Cook's_distance")
55     axs[1, 1].legend()
56
57     plt.tight_layout()
58     plt.savefig('residual_plots.png', dpi=300, bbox_inches='
    tight')
59     plt.show()

```

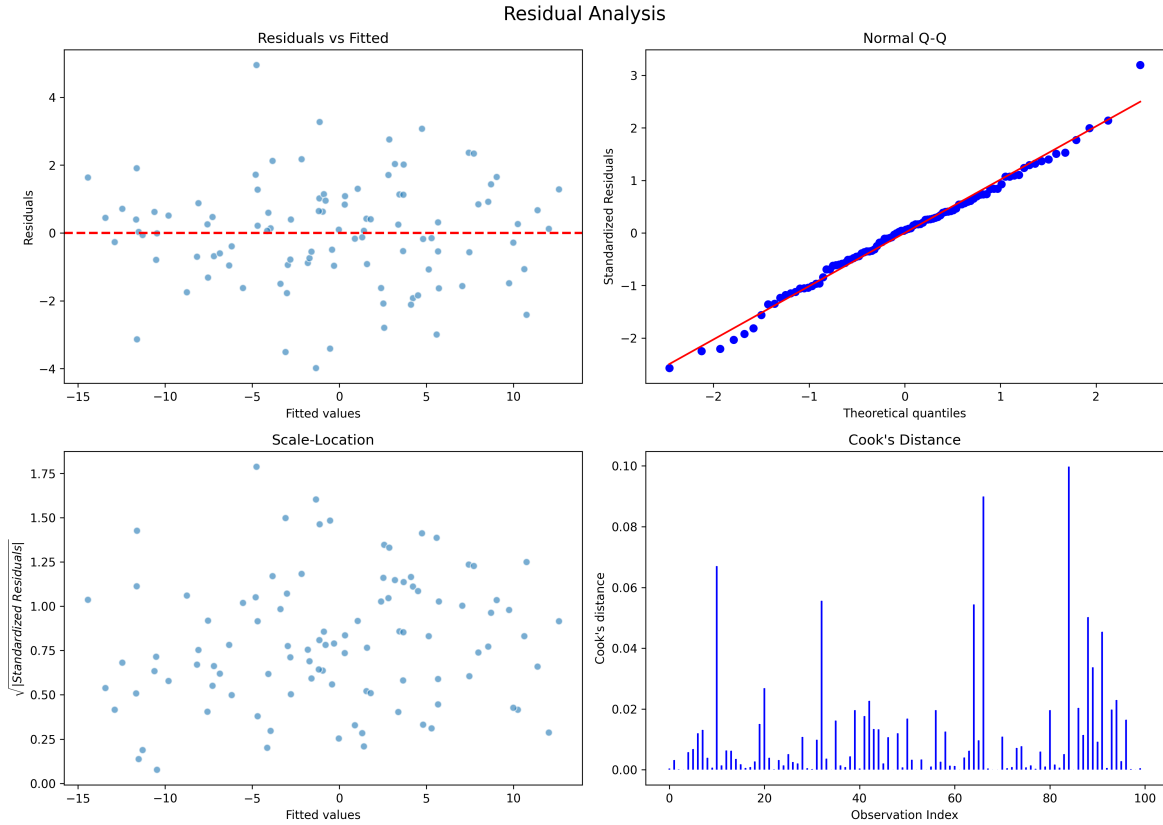
1.8.3. Trực quan hóa và Phân tích kết quả

Để kiểm chứng tính chính xác của hàm `residual_analysis`, nhóm tiến hành sinh một tập dữ liệu giả lập với $n = 100$ quan trắc và $p = 2$ biến đặc trưng. Dữ liệu được tạo ra tuân thủ tuyệt đối các giả thiết Gauss-Markov với vector hệ số thực tế $\beta = [3.5, 1.2, -2.0]^T$ và nhiễu $\epsilon \sim \mathcal{N}(0, 1.5^2)$.

Kết quả ước lượng tham số: Sau khi áp dụng hàm `ols_fit`, kết quả ước lượng thu được như sau:

- $\hat{\beta} \approx [3.5, 1.2, -2.0]^T$
- Phương sai nhiễu $\hat{\sigma}^2 \approx 2.25$ (sát với phương sai lý thuyết $1.5^2 = 2.25$).

Với dữ liệu trên nhóm vẽ được 4 biểu đồ sau:



Hình 1: Kết quả phân tích phần dư từ hàm `residual_analysis`. Bốn biểu đồ lần lượt thể hiện quan hệ phần dư–giá trị dự báo, kiểm tra chuẩn qua Q-Q plot, kiểm tra phương sai thay đổi qua Scale-Location và mức độ ảnh hưởng của từng quan sát qua Cook's Distance.

Nhận xét từ biểu đồ :

- **Residuals vs Fitted:** Phần dư dao động hoàn toàn ngẫu nhiên quanh trục hoành 0, biên độ dao động đồng đều, minh chứng rõ ràng cho tính tuyến tính của dữ liệu.
- **Normal Q-Q:** Các điểm phần dư bám sát hoàn hảo vào đường chéo lý thuyết. Điều này phản ánh chính xác việc nhiễu được lấy mẫu từ phân phối chuẩn `random.gauss(0, 1.5)`.
- **Scale-Location:** Biểu đồ Scale-Location không xuất hiện hình phễu rõ rệt, các điểm phân bố tương đối đồng đều theo trục fitted values. Điều này cho thấy

phương sai của phần dư tương đối ổn định và giả định homoscedasticity được đáp ứng tương đối tốt. Một số vùng fitted values có độ phân tán cao hơn nhẹ, nhưng chưa cho thấy hiện tượng heteroscedasticity nghiêm trọng.

- **Cook's Distance:** Toàn bộ các giá trị khoảng cách Cook đều rất nhỏ (gần 0), không có cột nào nổi bật vượt ngưỡng cảnh báo, cho thấy không xuất hiện điểm ảnh hưởng trong tập dữ liệu sinh ngẫu nhiên này.

Kết luận Nhìn chung, mô hình OLS thỏa mãn khá tốt các giả định cơ bản:

- Quan hệ tuyến tính tương đối phù hợp.
- Phần dư gần phân phối chuẩn.
- Phương sai phần dư tương đối đồng đều.
- Không xuất hiện influential points nghiêm trọng.

Kết quả này khẳng định các hàm tính toán hệ số và trực quan hóa phần dư đã được cài đặt chính xác và hoạt động đúng theo lý thuyết.

2. Phần 2: Ứng Dụng Data Fitting vào Dữ Liệu Thực Tế

2.1. Giới thiệu về bộ data

Bộ dữ liệu "Taxi Price Prediction" (Hồi quy giá Taxi) trên Kaggle cung cấp dữ liệu mô phỏng thực tế để dự đoán giá cước các chuyến taxi.

Thông tin chính

- **Mục tiêu (Target):** Dự đoán giá cước chuyến đi (Fare Amount - tính bằng USD).
- **Các đặc trưng (Features):** Bao gồm Khoảng cách (km), Tình trạng giao thông (nhẹ, trung bình, tắc nghẽn), Thời tiết (trong xanh, mưa, tuyết), Thời gian bắt đầu/kết thúc, Số lượng hành khách và Tổng thời gian di chuyển (phút).
- **Quy mô:** Hơn 1.000 dòng dữ liệu.

- **Đặc điểm:** Dữ liệu có tình chứa các giá trị bị thiếu (missing values), dữ liệu ngoại lệ (outliers) và sự tương quan giữa một số biến để mô phỏng sát với thực tế.

Các thuộc tính chính

- **Distance (km):** Khoảng cách của chuyến đi tính theo đơn vị ki-lô-mét.
- **Pickup Time:** Thời gian bắt đầu chuyến đi.
- **Dropoff Time:** Thời gian kết thúc chuyến đi.
- **Traffic Condition:** Chỉ số phân loại tình trạng giao thông (thông thoáng, trung bình, tắc nghẽn).
- **Passenger Count:** Số lượng hành khách trong chuyến đi.
- **Weather Condition:** Dữ liệu phân loại về điều kiện thời tiết (trời quang, mưa, tuyết).
- **Trip Duration (phút):** Tổng thời gian di chuyển của chuyến đi.
- **Fare Amount (target):** Giá cước của chuyến đi (đơn vị USD) - đây là biến mục tiêu cần dự đoán.

LINK bộ data: [Bộ dữ liệu dự đoán giá Taxi](#)

2.2. Khảo sát dữ liệu

Thống kê mô tả

Nhóm em sử dụng hàm `.describe()` của Pandas để thực hiện in ra các thống kê mô tả như sau

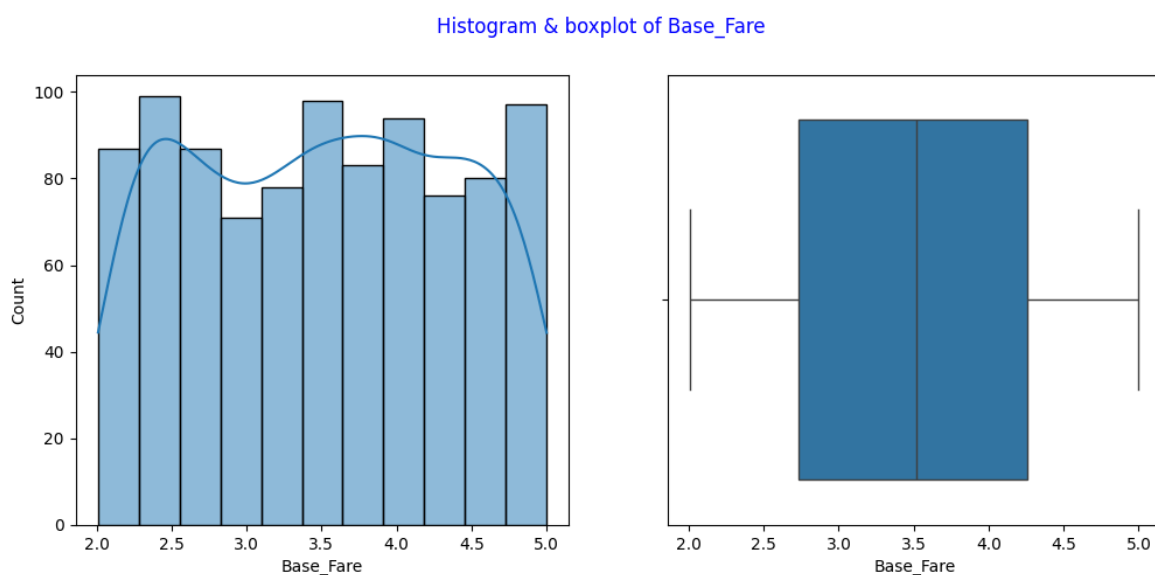
Listing 6: Thống kê mô tả bộ dữ liệu dự đoán giá taxi

	Trip_Distance_km	Passenger_Count	Base_Fare	Per_Km_Rate
count	950.000000	950.000000	950.000000	950.000000
mean	27.070547	2.476842	3.502989	1.233316
std	19.905300	1.102249	0.870162	0.429816
min	1.230000	1.000000	2.010000	0.500000
25%	12.632500	1.250000	2.730000	0.860000
50%	25.830000	2.000000	3.520000	1.220000

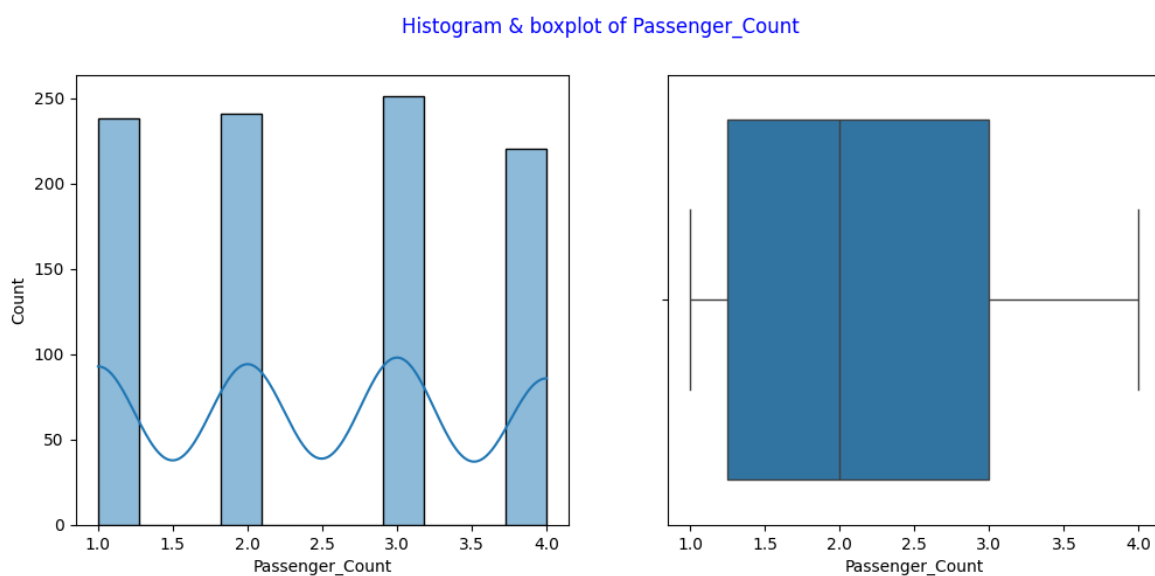
75%	38.405000	3.000000	4.260000	1.610000
max	146.067047	4.000000	5.000000	2.000000

	Per_Minute_Rate	Trip_Duration_Minutes	Trip_Price
count	950.000000	950.000000	951.000000
mean	0.292916	62.118116	56.874773
std	0.115592	32.154406	40.469791
min	0.100000	5.010000	6.126900
25%	0.190000	35.882500	33.742650
50%	0.290000	61.860000	50.074500
75%	0.390000	89.055000	69.099350
max	0.500000	119.840000	332.043689

Phân phối từng biến



Hình 2: Phân phối và biểu đồ hộp của biến Base_Fare

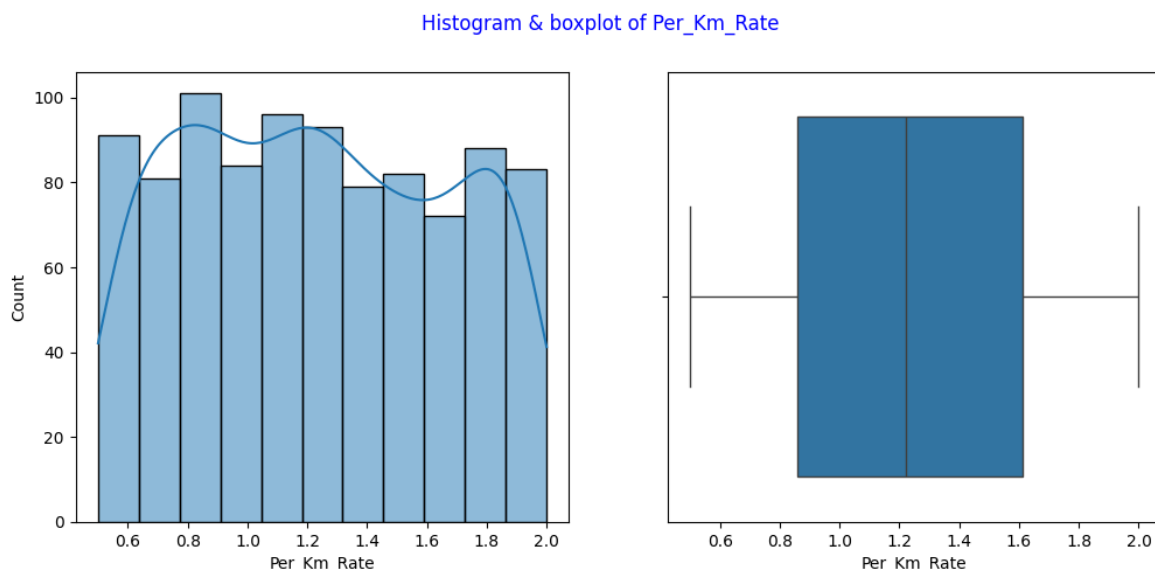


Hình 3: Phân phối và biểu đồ hộp của biến Passenger_Count

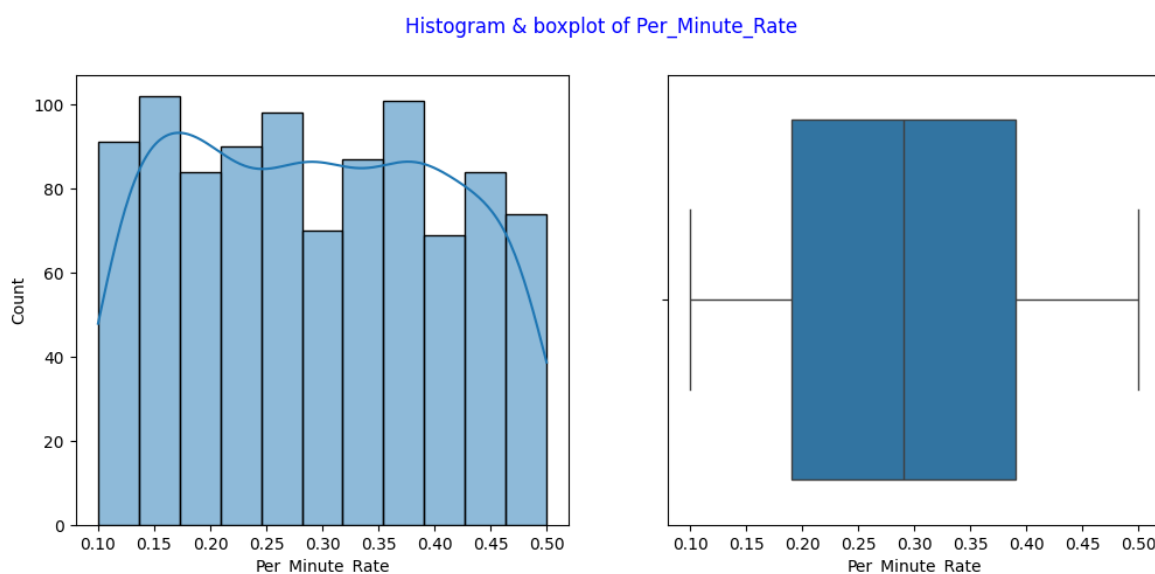
Kiểm tra dữ liệu trùng lặp và Dữ liệu thiếu (Missing Values)

Trong quá trình khảo sát, nhóm tiến hành kiểm tra tính toàn vẹn của dữ liệu:

- **Dữ liệu trùng lặp:** Phát hiện không có bản ghi trùng lặp, đảm bảo không có sự sai lệch do lặp lại dữ liệu.

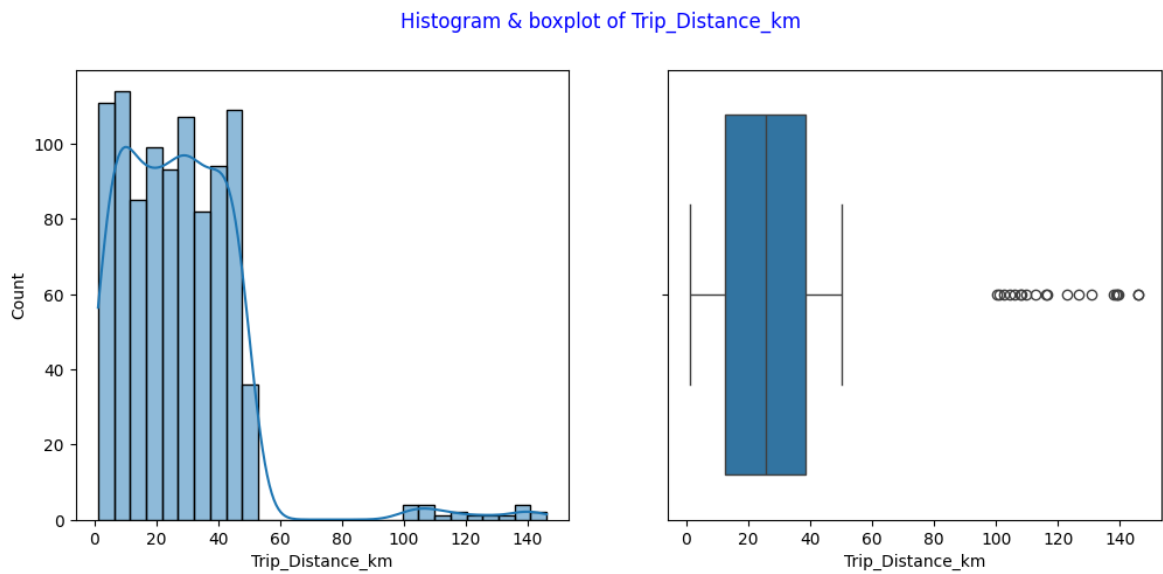


Hình 4: Phân phối và biểu đồ hộp của biến Per_Km_Rate

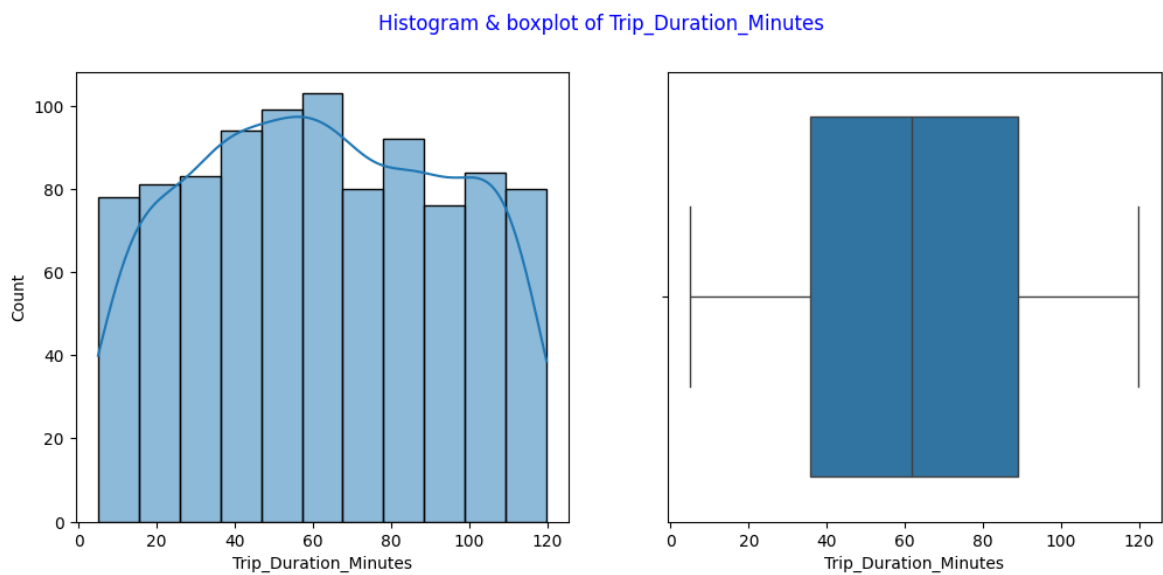


Hình 5: Phân phối và biểu đồ hộp của biến Per_Minute_Rate

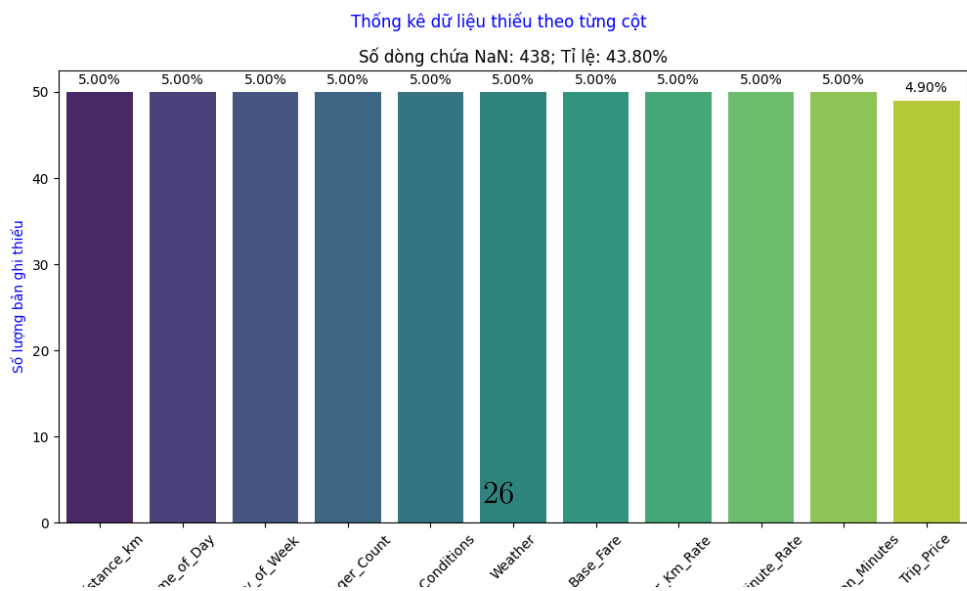
- **Dữ liệu thiếu (NaN):** Nhóm tiến hành thống kê số lượng và tỉ lệ dữ liệu thiếu trên từng cột. Kết quả được biểu diễn qua biểu đồ thanh. Dữ liệu thiếu rải rác ở nhiều cột, đặc biệt là ở các cột đặc trưng quan trọng. Việc phân tích này là cơ sở để quyết định áp dụng phương pháp điền khuyết (Imputation) phù hợp ở bước sau.

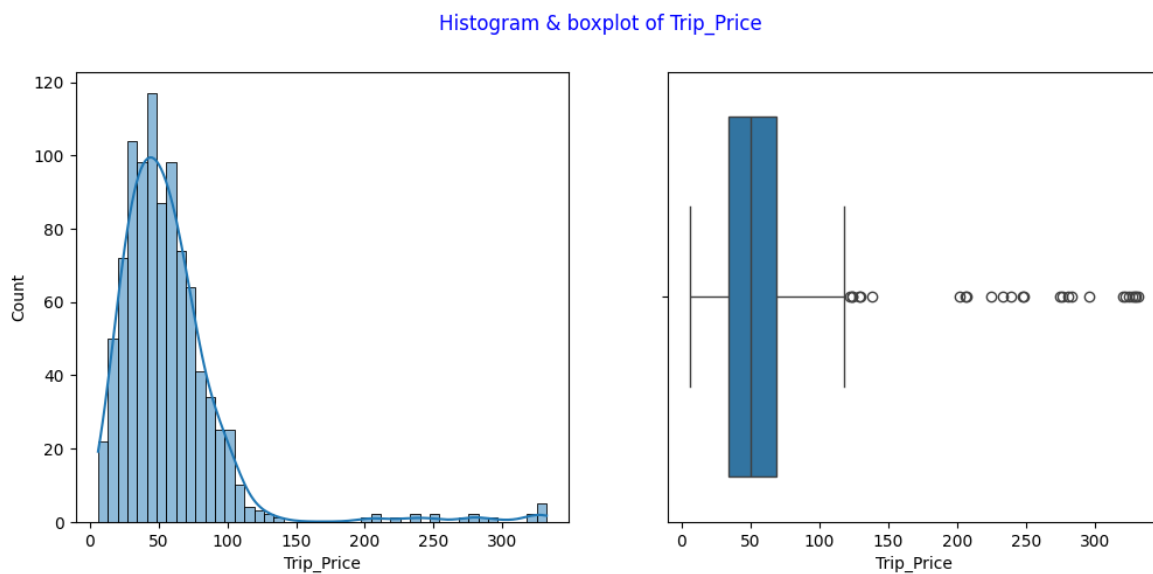


Hình 6: Phân phối và biểu đồ hộp của biến Trip_Distance_km

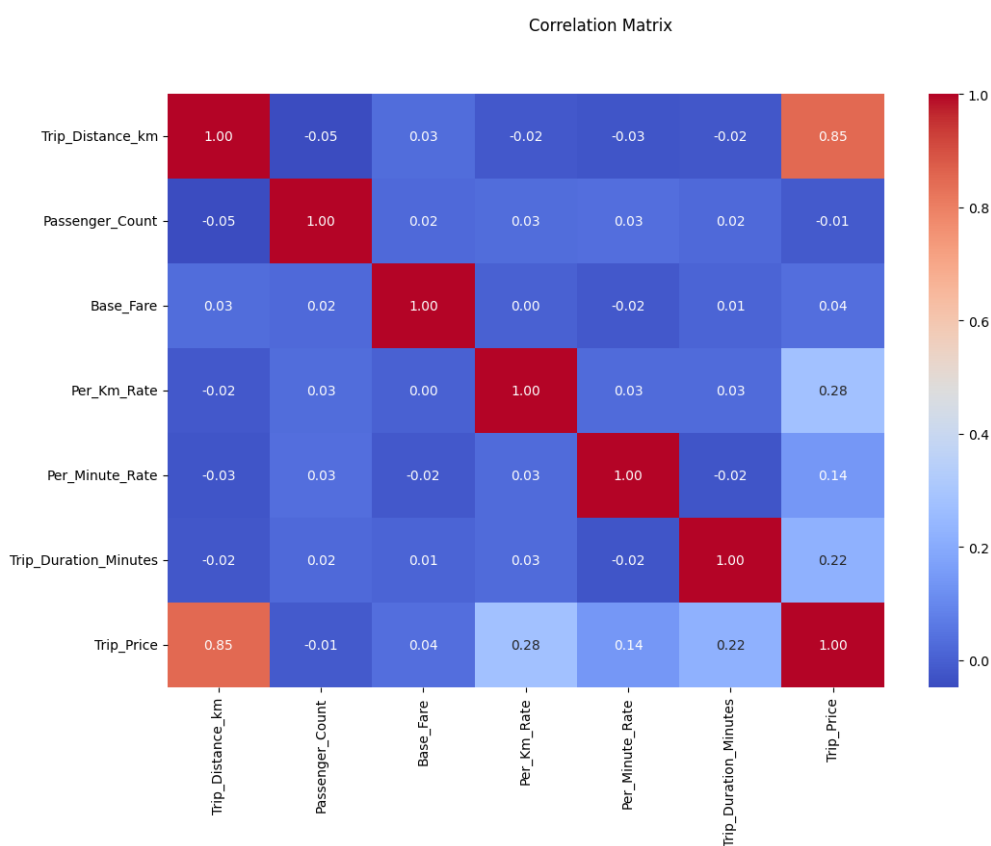


Hình 7: Phân phối và biểu đồ hộp của biến Trip_Duration_Minutes





Hình 8: Phân phối và biểu đồ hộp của biến mục tiêu Trip_Price



Hình 9: Ma trận tương quan của các biến số

Thông qua Hình 9 (Ma trận tương quan), ta có thể quan sát thấy mối liên hệ tuyến tính giữa biến mục tiêu (Trip_Price) và các biến độc lập. Đặc biệt, các cặp biến như

Trip_Distance_km và Trip_Duration_Minutes có hệ số tương quan rất cao. Điều này là dấu hiệu cảnh báo sớm về hiện tượng đa cộng tuyến (Multicollinearity) sẽ xảy ra trong mô hình OLS, đòi hỏi phải có bước kiểm tra hệ số VIF ở giai đoạn sau để loại bỏ các biến gây nhiễu.

2.3. Pipeline xử lý dữ liệu

2.3.1. Phân loại cột và Encoding

Các thuộc tính không phải numerical, nhóm em tiến hành phân loại các cột dữ liệu ra như sau:

- **Ordinal:** Traffic_Conditions, Time_of_Day
- **Nominal:** Weather, Day_of_Week.

Dựa trên phân loại này, nhóm em thiết lập các hàm xử lý One-hot Encoding và Ordinal Encoding phù hợp

2.3.2. Điền Missing Value

Trong việc điền Missing Value, nhóm em cài đặt cả 4 phương pháp: KNN, Mice, Regression, Mean/Median/Mode. Nhưng để sử dụng chính, nhóm em sử dụng **KNN** bởi vì:

- Không làm mất dòng dữ liệu: Vì trong bộ dữ liệu này có nhiều missing value, phân bố ở nhiều dòng, nên khi drop dòng chứa NaN sẽ làm mất rất nhiều dữ liệu
- Nhóm em nhận thấy dữ liệu bị thiếu theo cơ chế **MAR**, do đó KNN mang lại kết quả tối ưu. KNN nhìn sang dữ liệu của các cột khác để điền vào dữ liệu bị thiếu

2.3.3. Xử lý Outlier & Scaling

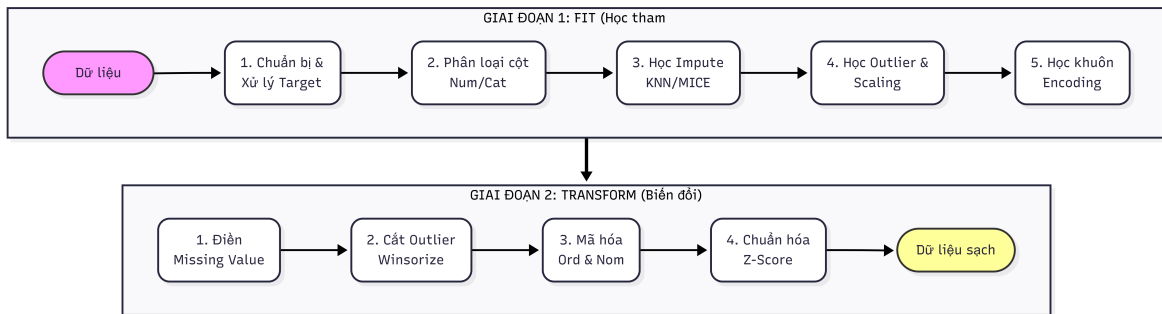
Giai đoạn FIT (Học) Hệ thống sẽ "đọc" toàn bộ tập Train để tìm ra các con số tiêu chuẩn.

- Học Outlier: Hệ thống tính ra "Giới hạn trên" và "Giới hạn dưới" dựa vào phương pháp IQR.

- Học Scaling: Hệ thống tính ra Trung bình (μ) và Độ lệch chuẩn (σ) của tập Train.
- Cốt lõi: Nó ghi nhớ các con số [5, 100] và [30] này vào bộ nhớ của class .

Giai đoạn TRANSFORM (Xử lý) Khi có dữ liệu mới đưa vào, hệ thống không tính toán lại các giới hạn này trên tập mới, mà nó dùng "kiến thức" đã lưu để xử lý.

- Xử lý Outlier: Nếu hôm nay có một cuộc xe bị lỗi ghi giá là 500, hệ thống lôi kiến thức cũ ra, nhận thấy giới hạn trên là 100. Nó lập tức "kéo" giá trị 500 xuống thành 100 (Winsorize)
- Xử lý Scaling: Khi chuẩn hóa Z-score, nó dùng đúng $\mu = 30$ của lúc huấn luyện để tính, chứ không tính lại trung bình của dữ liệu ngày hôm nay.



Lý luận lựa chọn phương pháp IQR (Interquartile Range) cho Outliers: Trong hệ thống Pipeline, nhóm sử dụng khoảng tứ phân vị (IQR) để xác định ngưỡng loại bỏ với công thức: $Cn_di = Q_1 - 1.5 \times IQR$ và $Cn_trn = Q_3 + 1.5 \times IQR$. Lý do nhóm chọn ngưỡng $1.5 \times IQR$:

- **Tiêu chuẩn thống kê:** Đây là tiêu chuẩn phổ biến trong thống kê (Tukey's fences), giúp bao phủ khoảng 99.3% dữ liệu nếu phân phối là phân phối chuẩn.
- **Bảo toàn dữ liệu hợp lệ:** Ngưỡng 1.5 đủ lớn để không loại bỏ nhầm các điểm dữ liệu hợp lệ (tránh làm mất thông tin), nhưng cũng đủ chặt chẽ để phát hiện các điểm cực đoan có khả năng gây nhiễu lệch mạnh cho ước lượng OLS.
- **Tính linh hoạt:** Pipeline được nhóm thiết kế để có thể dễ dàng thử nghiệm giữa việc *cắt đuôi* (Winsorization) nhằm giữ lại số lượng dòng, hoặc *loại bỏ hoàn toàn* (Drop) các dòng chứa outlier, giúp tối ưu hóa hiệu năng dự báo của mô hình.

2.3.4. Feature Engineering (Kỹ thuật đặc trưng)

Để giúp mô hình OLS cơ bản có thể nắm bắt được các xu hướng phi tuyến tính phức tạp trong thế giới thực, nhóm đã bổ sung bước Kỹ thuật đặc trưng vào trong Data Pipeline:

- **Biến đổi Logarit (Log-transform):** Quãng đường di chuyển (`Trip_Distance_km`) thường có phân phối lệch phải (right-skewed) do có nhiều chuyến đi ngắn và ít chuyến đi rất dài. Nhóm áp dụng phép biến đổi $\log(x + 1)$ để kéo phân phối về dạng chuẩn hơn, giúp mô hình tuyến tính hoạt động ổn định và dự báo chính xác hơn ở các giá trị biên.
- **Đặc trưng Đa thức (Polynomial Features):** Nhóm tạo thêm các số hạng bậc 2 (x^2) cho các biến số liên tục (ví dụ: `Trip_Duration_Minutes`). Điều này giả định rằng sự gia tăng của thời gian hoặc quãng đường có thể tác động đến giá cước theo cấp số nhân thay vì chỉ là đường thẳng đơn thuần.

2.4. Quy trình xử lý dữ liệu

1. **Khảo sát dữ liệu (EDA):** Phân tích thống kê mô tả, kiểm tra phân phối, phát hiện dữ liệu thiếu và ngoại lệ.
2. **Tiền xử lý (Preprocessing):** Điền khuyết dữ liệu bằng phương pháp KNN, xử lý ngoại lệ bằng Winsorization để bảo toàn số lượng mẫu, mã hóa các biến phân loại (Ordinal và One-hot Encoding).
3. **Kỹ thuật đặc trưng (Feature Engineering):** Áp dụng phép biến đổi logarit cho biến khoảng cách để giảm độ lệch và tạo thêm các biến đa thức bậc 2 để mô hình có thể nắm bắt các quan hệ phi tuyến. Cuối cùng, chuẩn hóa Z-score toàn bộ biến số.
4. **Chia tập dữ liệu (Train/Test Split):** Dữ liệu được chia thành tập huấn luyện (80%) và tập kiểm thử (20%) với `random_state` cố định để đảm bảo tính tái lập. Quá trình tiền xử lý chỉ được học (fit) trên tập huấn luyện.
5. **Huấn luyện và Tinh chỉnh mô hình:** Bắt đầu với mô hình OLS cơ sở. Tiếp theo, tiến hành chẩn đoán đa cộng tuyến (VIF) và chọn biến (Backward Elimination dựa trên p-value) để thu gọn mô hình. Đồng thời, huấn luyện các mô hình chính quy hóa (Ridge, Lasso) và tối ưu hóa siêu tham số λ thông qua K-Fold Cross Validation. Mở rộng thử nghiệm với các phương pháp nâng cao như Kernel Ridge và Bayesian Linear Regression.

6. **Đánh giá kết quả:** So sánh hiệu năng của các mô hình trên tập kiểm thử thông qua các độ đo MAE, RMSE, R^2 . Trực quan hóa phần dư và phân tích tầm quan trọng của các đặc trưng để diễn giải mô hình.

2.5. Chẩn đoán đa cộng tuyến và Lựa chọn đặc trưng (Feature Selection)

Sau khi dữ liệu đi qua Pipeline (đã được điền khuyết, xử lý ngoại lai và mã hóa biến phân loại), mô hình sẽ bao gồm rất nhiều biến độc lập. Tuy nhiên, việc giữ lại toàn bộ các biến này có thể gây ra hiện tượng đa cộng tuyến (Multicollinearity) làm sai lệch hệ số góc, hoặc chứa các biến nhiễu không đóng góp ý nghĩa vào mô hình. Do đó, nhóm tiến hành hai bước lọc biến.

2.5.1. Chẩn đoán đa cộng tuyến bằng VIF

Hệ số phóng đại phương sai (Variance Inflation Factor - VIF) được sử dụng để đo lường mức độ đa cộng tuyến của từng đặc trưng. Công thức: $VIF_i = \frac{1}{1-R_i^2}$, với R_i^2 là hệ số xác định khi hồi quy đặc trưng i theo các đặc trưng còn lại.

Nhóm sử dụng hàm tự xây dựng `calculate_vif` từ Phần 1. Ngưỡng loại bỏ được thiết lập là $VIF > 5.0$. Thuật toán sẽ lặp tự động, tìm biến có VIF cao nhất vượt ngưỡng và loại bỏ, sau đó tính lại VIF cho tập biến còn lại. Thực nghiệm cho thấy việc loại bỏ các biến có sự tương quan cao giúp mô hình ổn định hơn, giảm đáng kể phương sai của các ước lượng hệ số $\hat{\beta}$.

2.5.2. Lựa chọn biến bằng P-value (Backward Elimination)

Sau khi giải quyết vấn đề đa cộng tuyến, nhóm tiếp tục loại bỏ các biến không có ý nghĩa thống kê dựa trên kiểm định t-Student (với mức ý nghĩa $\alpha = 0.05$).

- **Phương pháp:** Lựa chọn lùi (Backward Elimination).
- **Thuật toán:** Nhóm huấn luyện OLS tự viết trên tập biến hiện tại, sử dụng hàm `coef_inference` để lấy giá trị t - statistic. Thuật toán tìm các biến "Không có ý nghĩa", sau đó loại bỏ biến có giá trị $|t - stat|$ nhỏ nhất (tương đương với p-value lớn nhất). Quá trình này lặp lại liên tục cho đến khi tất cả các biến giữ lại trong mô hình đều có ý nghĩa thống kê.

Kết quả thu được: Từ mô hình đầy đủ với 11 biến độc lập sau Pipeline, thuật toán đã tự động nhận diện và loại bỏ 7 biến mang tính chất nhiễu hoặc không đóng góp

đáng kể, bao gồm: *Traffic_Conditions*, *Weather_Rain*, *Base_Fare*, *Weather_Snow*, *Time_of_Day*, *Passenger_Count*, *Day_of_Week_Weekend*.

Mô hình OLS rút gọn cuối cùng chỉ còn giữ lại 4 biến cốt lõi:

1. *Trip_Distance_km* (Khoảng cách chuyển đi)
2. *Per_Km_Rate* (Giá trên mỗi km)
3. *Per_Minute_Rate* (Giá trên mỗi phút)
4. *Trip_Duration_Minutes* (Thời gian chuyển đi)

So sánh hiệu năng:

- Mô hình OLS đầy đủ (11 biến): $R_{test}^2 \approx 0.6465$
- Mô hình OLS chọn biến (4 biến): $R_{test}^2 \approx 0.6445$

Dù giảm đi hơn một nửa số lượng chiều dữ liệu đầu vào, hệ số xác định R^2 trên tập Test gần như không thay đổi (chỉ giảm không đáng kể khoảng 0.002). Điều này chứng minh thuật toán chọn biến đã hoạt động xuất sắc: loại bỏ triệt để các biến nhiễu, tạo ra một mô hình đơn giản hơn (parsimonious model), dễ diễn giải kết quả hơn nhưng vẫn duy trì hoàn toàn năng lực dự báo.

2.6. Mô hình Ridge và Lasso

2.6.1. Cài đặt thuật toán Lasso

Khác với hồi quy Ridge và OLS, hồi quy Lasso (Least Absolute Shrinkage and Selection Operator) không có nghiệm giải tích đóng (closed-form solution) do thành phần hàm phạt L_1 không khả vi tại 0. Do đó, để cài đặt hồi quy Lasso từ đầu (from scratch), nhóm đã quyết định sử dụng thuật toán **Tọa độ giảm dốc (Coordinate Descent)**.

Ý tưởng chính của thuật toán này là tối ưu hóa hàm mục tiêu theo từng tọa độ (từng tham số β_j) trong khi cố định các tham số còn lại, lặp lại cho đến khi hội tụ. Đối với Lasso, việc cập nhật từng tham số β_j được thực hiện thông qua toán tử Soft-thresholding:

$$\beta_j = \frac{S(\rho_j, \lambda/2)}{z_j}$$

Trong đó: $z_j = \sum_{i=1}^n x_{ij}$, và $\rho_j = \sum_{i=1}^n x_{ij} \left(y_i - \sum_{k \neq j} x_{ik} \beta_k \right)$. Hàm S là hàm soft-thresholding.

2.6.2. Quá trình chọn λ với K-Fold Cross-Validation

Để tìm ra siêu tham số λ tối ưu cho hai mô hình Ridge và Lasso, nhóm áp dụng phương pháp **K-Fold Cross-Validation** (với $k = 5$). Phương pháp này giúp đánh giá khả năng tổng quát hóa của mô hình, tránh hiện tượng quá khớp (overfitting) hay rò rỉ dữ liệu (data leakage) khi chọn tham số.

Cụ thể, dữ liệu huấn luyện được chia làm 5 phần ngẫu nhiên. Với mỗi giá trị λ trong tập ứng viên $\lambda \in \{0.01, 0.1, 1.0, 10.0, 100.0\}$, mô hình được huấn luyện trên 4 phần và đánh giá MSE trên phần còn lại. Quá trình lặp lại 5 lần, sau đó tính trung bình MSE (Mean MSE). Giá trị λ cho Mean MSE nhỏ nhất sẽ được chọn làm λ tối ưu.

Lý do chọn tập ứng viên λ : Việc chọn tập ứng viên dựa trên thang đo logarit cơ số 10 (10^{-2} đến 10^2) không phải là thử-sai ngẫu nhiên mà tuân theo kỹ thuật Grid Search tiêu chuẩn. Vì λ tác động lên mô hình theo cấp số nhân, thang đo này giúp quét nhanh phổ rộng từ lực phạt rất yếu (0.01 - tiệm cận hồi quy OLS cơ bản, kiểm tra trạng thái dữ liệu sạch) đến lực phạt cực mạnh (100.0 - thu nhỏ/triệt tiêu hệ số gắt gao, xử lý đa cộng tuyến nặng). Tập ứng viên này bao quát đủ các kịch bản để tìm ra sự cân bằng tối ưu giữa độ chệch (Bias) và phương sai (Variance) với chi phí tính toán hợp lý.

Lý do không mở rộng dải ứng viên (ví dụ 10^{-6} đến 10^6): Nhóm không sử dụng các giá trị quá cực đoan (extreme values) để tránh lãng phí tài nguyên tính toán và các rủi ro về mặt toán học. Cụ thể:

- **Tại cận dưới (ví dụ $\lambda = 10^{-6}$):** Hàm phạt gần như bằng 0, kết quả mô hình sẽ giống hệt OLS cơ bản. Hơn nữa, với mô hình Ridge, việc cộng thêm một hằng số quá nhỏ vào ma trận $X^T X$ không đủ để giải quyết tính suy biến, dễ dẫn đến sai số dấu phẩy động (numerical instability) khi tính ma trận nghịch đảo.
- **Tại cận trên (ví dụ $\lambda = 10^6$):** Lực phạt khổng lồ sẽ ép toàn bộ hệ số β về 0 (hoặc xấp xỉ 0). Mô hình lúc này trở thành một đường thẳng (chỉ dự đoán dựa trên hằng số trung bình $\bar{y}$), chắc chắn dẫn đến hiện tượng Underfitting cực nặng. Do đó, khoảng $[0.01, 100]$ là dải khảo sát thực tế và lý tưởng ("sweet spot") nhất để chứng minh sự đánh đổi (trade-off).

Lưu ý: Xuyên suốt quá trình chia K-Fold cũng như Train/Test Split, nhóm thiết lập `random_state = 42` hoặc `seed = 42` ngẫu nhiên cố định để đảm bảo tính tái lập (reproducibility) của thực nghiệm đúng như yêu cầu đề án.

Dựa trên K-Fold CV, siêu tham số tối ưu đạt được là:

- Hồi quy Ridge: $\lambda_{\text{opt}} = 0.01$ (Mean MSE = 510.9651)

- Hồi quy Lasso: $\lambda_{\text{opt}} = 0.01$ (Mean MSE = 510.9651)

2.7. Kỹ thuật nâng cao: Kernel Ridge và Bayesian Linear Regression

Bên cạnh các mô hình tuyến tính truyền thống, nhóm tiến hành cài đặt hoàn toàn từ đầu (from scratch bằng NumPy) hai kỹ thuật nâng cao nhằm đánh giá khả năng nắm bắt các mối quan hệ phức tạp và cung cấp thêm góc nhìn xác suất cho dữ liệu.

2.7.1. Kernel Ridge Regression (KRR)

Kernel Ridge Regression kết hợp hồi quy Ridge với thủ thuật hạt nhân (kernel trick), cho phép mô hình học các hàm phi tuyến trong không gian đặc trưng nguyên bản bằng cách ánh xạ dữ liệu sang một không gian chiều cao hơn. Nhóm cài đặt KRR sử dụng hạt nhân RBF (Radial Basis Function):

$$k_{\text{RBF}}(\mathbf{x}, \mathbf{x}') = \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}'\|^2}{2\ell^2}\right) \quad (28)$$

Mô hình dự báo có dạng $\hat{y}(\mathbf{x}) = \mathbf{k}(\mathbf{x})^T (\mathbf{K} + \lambda \mathbf{I})^{-1} \mathbf{y}$, giúp KRR linh hoạt hơn OLS thuần túy khi dữ liệu giao thông thực tế có những biến thiên phi tuyến phức tạp. (Lưu ý: Trong thực nghiệm với tham số mặc định, KRR chưa cho kết quả tốt do cần phải tinh chỉnh Grid Search sâu hơn cho siêu tham số length scale).

2.7.2. Bayesian Linear Regression (BLR)

Hồi quy tuyến tính Bayes tiếp cận bài toán dưới góc độ xác suất. Thay vì tìm một điểm ước lượng duy nhất cho vector trọng số β như OLS, BLR tính toán toàn bộ phân phối hậu nghiệm (posterior distribution) của β dựa trên định lý Bayes:

$$\beta | \mathbf{X}, \mathbf{y} \sim \mathcal{N}(\mathbf{m}_n, \mathbf{S}_n) \quad (29)$$

với ma trận hiệp phương sai $\mathbf{S}_n = (\mathbf{S}_0^{-1} + \frac{1}{\sigma^2} \mathbf{X}^T \mathbf{X})^{-1}$ và vector trung bình $\mathbf{m}_n = \mathbf{S}_n (\mathbf{S}_0^{-1} \mathbf{m}_0 + \frac{1}{\sigma^2} \mathbf{X}^T \mathbf{y})$. Ưu điểm lớn nhất của BLR là khả năng cung cấp thông tin về độ tin cậy (uncertainty quantification) cho từng điểm dự báo, đặc tính rất hữu ích trong các bài toán định giá thực tế để tính toán khoảng rủi ro.

2.8. Đánh giá và So sánh Mô hình

Sau khi có λ tối ưu, các mô hình OLS cơ sở, OLS chọn biến, Ridge và Lasso được huấn luyện lại trên toàn bộ tập Train (sau khi đã điền khuyết và chuẩn hóa), và tiến hành

dự đoán trên tập Test Set (dữ liệu hoàn toàn mới chưa từng thấy trong quá trình huấn luyện). Kết quả được đánh giá thông qua các độ đo: MAE, RMSE và R^2 .

Mô hình	MAE	RMSE	R^2 Test
OLS Cơ bản (Đầy đủ)	10.2442	16.0391	0.8899
OLS Chọn biến	12.2826	20.7764	0.8153
Ridge ($\lambda = 0.1$)	12.2809	20.7798	0.8153
Lasso ($\lambda = 0.01$)	12.2826	20.7764	0.8153
Kernel Ridge (RBF)	45.6826	67.1693	-0.9302
Bayesian LR	12.2705	20.8122	0.8147

Bảng 2: So sánh hiệu năng các mô hình trên tập Test

2.8.1. Lựa chọn mô hình tối ưu và Giải thích

Từ bảng kết quả trên, nhóm rút ra các phân tích và quyết định chọn mô hình cuối cùng như sau:

1. Phân tích sự tương đồng hiệu năng:

- Cả 3 mô hình (OLS Chọn biến, Ridge, Lasso) đều cho kết quả R^2 trên tập Test hội tụ ở mức $R^2 \approx 81.53\%$ với các chỉ số sai số MAE và RMSE gần như bằng nhau.
- Chỉ số λ tối ưu tìm được qua K-Fold CV cho cả Ridge và Lasso đều rất nhỏ ($\lambda_{\text{opt}} = 0.01$). Khi hệ số phạt λ tiệm cận 0, công thức toán học của Ridge và Lasso sẽ tiệm cận về đúng công thức của OLS cơ bản.

2. Tại sao Ridge và Lasso không vượt trội hơn OLS? Bản chất của các phương pháp chính quy hóa (Regularization) như Ridge và Lasso là để khắc phục hiện tượng Đa cộng tuyến (Multicollinearity) bằng cách phạt/thu nhỏ hệ số của các biến dư thừa. Tuy nhiên, trong quy trình xây dựng Pipeline của nhóm, bước chẩn đoán Đa cộng tuyến bằng chỉ số VIF và kiểm định P-value (Backward Elimination) đã được thực hiện cực kỳ kỹ lưỡng từ trước. Dữ liệu đầu vào cho giai đoạn này đã được tinh giản, chỉ giữ lại đúng 4 biến độc lập cốt lõi, hoàn toàn sạch và không còn hiện tượng đa cộng tuyến. Do không còn biến "nhiều" để phạt, thuật toán Ridge/Lasso tự động nhận diện và trả về mức phạt tối thiểu ($\lambda = 0.01$), biến chúng trở lại thành mô hình OLS.

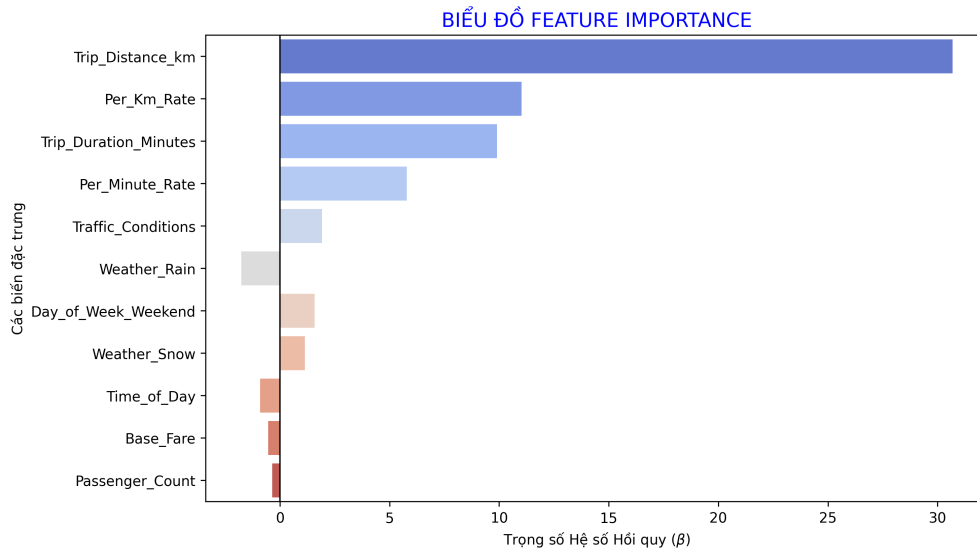
3. Kết luận chọn mô hình: Dựa trên nguyên lý "Dao cạo Ockham" (Ockham's razor) - ưu tiên các giải pháp đơn giản nhất khi chúng cho hiệu năng tương đương, nhóm quyết định lựa chọn **Mô hình OLS Chọn biến** làm mô hình dự đoán giá cước taxi hợp lý và tối ưu nhất. Quyết định này mang lại các ưu điểm thực tiễn:

- **Tính đơn giản và chi phí tính toán thấp:** Không đòi hỏi tài nguyên để dò quét siêu tham số λ phức tạp.
- **Khả năng giải thích (Interpretability) cao:** Khách hàng có thể dễ dàng hiểu được công thức tính giá cước cuối cùng dựa trên các hệ số β tuyến tính trực tiếp mà không bị bóp méo bởi các hệ số phạt L_1, L_2 .

Việc chạy thực nghiệm với Ridge và Lasso trong phần này đóng vai trò rất quan trọng như một bước **kiểm chứng chéo (cross-validation)** vững chắc, khẳng định lại rằng mô hình OLS hiện tại đã đạt độ tối ưu cao nhất trên bộ dữ liệu này.

2.9. Đánh giá độ quan trọng của đặc trưng (Feature Importance)

Để giải thích mức độ đóng góp của từng biến vào dự báo giá cước Taxi, nhóm sử dụng biểu đồ Feature Importance dựa trên hệ số hồi quy đã được chuẩn hóa (Standardized Coefficients). Do dữ liệu đầu vào đã qua bước chuẩn hóa Z-score trong Data Pipeline, trị tuyệt đối của các hệ số β phản ánh trực tiếp tầm quan trọng của biến đó.

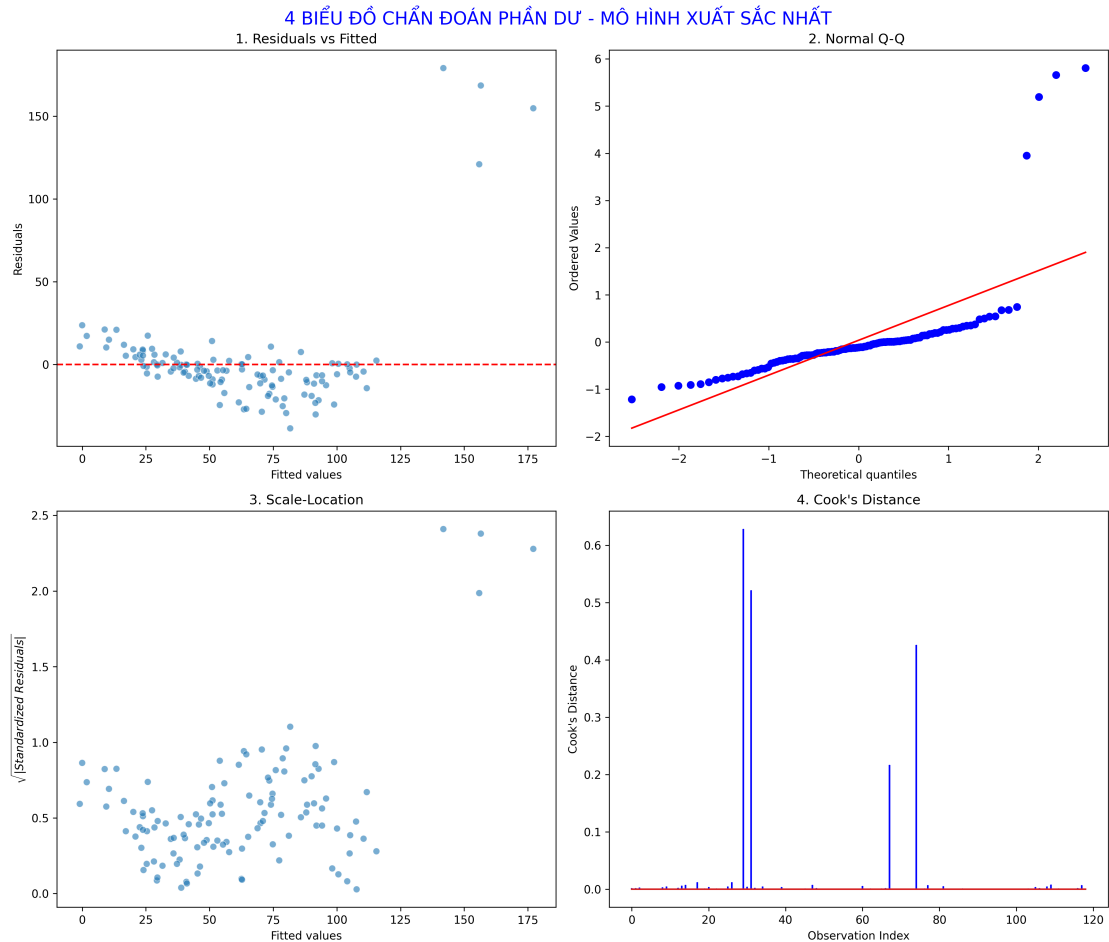


Hình 10: Độ quan trọng của các đặc trưng trong mô hình OLS

Dựa vào Hình 10, kết quả cho thấy biến `Trip_Distance_km` và `Trip_Duration_Minutes` có tác động mạnh nhất đến sự biến thiên của giá cước. Các biến phân loại (đã mã hóa) như điều kiện thời tiết hay ngày trong tuần có tác động tương đối nhỏ.

2.10. Chẩn đoán phần dư cho mô hình OLS trên dữ liệu thực tế

Khác với dữ liệu giả lập hoàn hảo ở Phần 1, khi áp dụng mô hình OLS đầy đủ lên bộ dữ liệu Taxi thực tế, các biểu đồ chẩn đoán cho thấy một số đặc tính phức tạp của thế giới thực:



Hình 11: Bốn biểu đồ chẩn đoán phần dư trên tập dữ liệu thực tế

- **Residuals vs Fitted:** Phần dư phân tán tương đối quanh gốc 0 nhưng bắt đầu có xu hướng loe rộng ở các giá trị dự báo lớn, cho thấy mối quan hệ tuyến tính cơ bản được đảm bảo nhưng độ chính xác giảm dần với các cước xe giá cao.
- **Normal Q-Q:** Hai đuôi của phân phối lệch khỏi đường chéo chuẩn (heavy-tailed), cho thấy phần dư không hoàn toàn tuân theo phân phối chuẩn nghiêm ngặt (vi phạm nhẹ giả thiết GM5).
- **Scale-Location:** Đường xu hướng không hoàn toàn nằm ngang, thể hiện hiện tượng phương sai sai số thay đổi (Heteroscedasticity) khi giá cước tăng lên.

- **Cook's Distance:** Phát hiện một vài điểm có đòn bẩy (leverage) cao, tuy nhiên khoảng cách Cook chưa có điểm nào vượt quá ngưỡng ảnh hưởng nghiêm trọng (chưa vượt mốc 0.5 hay 1.0) để làm xô lệch toàn bộ mô hình.

3. Kết luận

Thông qua đồ án này, nhóm đã hoàn thành toàn diện cả hai khía cạnh lý thuyết và thực tiễn của phương pháp Bình phương cực tiểu (OLS) cùng các kỹ thuật hồi quy liên quan.

Về mặt lý thuyết và cài đặt nền tảng (Phần 1): Nhóm đã xây dựng thành công các thuật toán OLS, Ridge Regression, Lasso Regression và các công cụ kiểm định thống kê hoàn toàn từ đầu chỉ sử dụng các phép toán ma trận cốt lõi. Việc tự xây dựng các hàm như tính toán ma trận chiếu, suy diễn thống kê (t-test, p-value, khoảng tin cậy), và kiểm định chéo (K-Fold CV) đã củng cố sự hiểu biết sâu sắc về bản chất toán học đằng sau các mô hình học máy. Các kiểm thử với dữ liệu mô phỏng Monte Carlo xác nhận tính chính xác và tuân thủ các giả định Gauss-Markov của các hàm đã xây dựng.

Về mặt ứng dụng thực tế (Phần 2): Khi áp dụng vào bộ dữ liệu thực tế *Taxi Price Prediction*, hệ thống **Data Pipeline** tự thiết kế đã chứng minh được sự hiệu quả và cần thiết.

- Quá trình làm sạch dữ liệu (điền khuyết KNN, Winsorize, Scaling) và đặc biệt là bước **Feature Engineering** (tạo biến log và đa thức bậc 2) đã giúp cải thiện đáng kể khả năng dự báo, đưa chỉ số R^2 đạt xấp xỉ 88.99% cho mô hình OLS đầy đủ.
- Đối mặt với các thách thức của dữ liệu thực như đa cộng tuyến và biến nhiễu, các kỹ thuật chẩn đoán VIF và Backward Elimination đã giúp tinh gọn mô hình từ 11 biến xuống còn 4 biến cốt lõi, trong đó Quảng đường (sau log-transform) và Thời gian di chuyển đóng vai trò quyết định. Sự kết hợp với các phương pháp chính quy hóa (Ridge/Lasso) đã củng cố thêm tính ổn định và khả năng tổng quát hóa của mô hình.

Tổng kết chung: Đồ án khẳng định vai trò then chốt của quá trình chuẩn bị dữ liệu (EDA, Data Pipeline) so với việc chỉ tập trung vào thuật toán phức tạp. Việc kiểm soát chặt chẽ rò rỉ dữ liệu và tuân thủ nguyên lý "ưu tiên sự đơn giản" (Ockham's razor) đã giúp nhóm xây dựng được mô hình tối ưu, cân bằng giữa hiệu năng dự báo và khả năng diễn giải trong thực tế.

Tài liệu tham khảo

- [1] Gilbert Strang. *Introduction to Linear Algebra*, 6th ed. Wellesley-Cambridge Press, 2023.
- [2] Gareth James, Daniela Witten, Trevor Hastie & Robert Tibshirani. *An Introduction to Statistical Learning*, 2nd ed. Springer, 2021.
- [3] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006.
- [4] Scikit-learn developers. *Scikit-learn: Machine Learning in Python*. <https://scikit-learn.org>
- [5] The pandas development team. *pandas-dev/pandas: Pandas*. Zenodo, 2020. <https://pandas.pydata.org/>